DATA STRUCTURES AND ALGORITHMS · 2026 ASSIGNMENT

Shortest Routes and Minimum Spanning Tree on the Malawi Road Network

A weighted graph of 25 Malawian towns and 38 roads, solved with Dijkstra's algorithm, Kruskal's algorithm and Prim's algorithm, implemented in C++ and deployed as a web application.

Group members

#	Name	Registration number
1	[Member 1 full name]	[Registration number]
2	[Member 2 full name]	[Registration number]
3	[Member 3 full name]	[Registration number]

Source code	https://github.com/mse23-mkapolo-cpu/DSA
Live demo	https://mse23-mkapolo-cpu.github.io/DSA/
Date	November 2026

Contents

1. The graph: Malawi road network (Q1)
2. Drawing and description of the graph (Q2)
3. How Dijkstra's algorithm works on the graph (Q3)
4. Implementation (Q4)
5. Testing and dry run (Q5)
6. Does the graph have an MST? (Q6)
7. Which MST algorithm and why (Q7)
8. Deployment (Q8)
9. Appendix: source code

1. The graph: Malawi road network

Q1 · 15 marks

Scenario. The graph models the main road network that links district centres and major towns in all three regions of Malawi. Each **vertex** is a town and each **edge** is a road that connects two towns directly, without passing through another town in the graph. The **weight** of an edge is the road distance in kilometres.

The graph is **undirected** because Malawian trunk roads such as the M1, M5 (lakeshore road) and M3 carry traffic both ways, so the distance from A to B equals the distance from B to A. It is **weighted** because the roads have different lengths, and it is **simple** (no self loops, at most one road between two towns).

Distances are approximate, rounded to the nearest 5 km where possible, based on the main routes (for example M1 Lilongwe–Dedza–Ntcheu–Blantyre, M5 Nkhata Bay–Nkhotakota–Salima, M3 Liwonde–Mangochi). They are realistic enough for the algorithms to make the same kind of trade-offs a real traveller would make, which is the purpose of the exercise.

25	38	3,854	3
vertices (towns)	edges (roads)	km of road in total	regions covered

Formally $G = (V, E, w)$ with $|V| = 25$, $|E| = 38$ and $w : E \to \mathbb{N}$ (km). The full data is in `data/malawi_roads.csv` in the repository, one road per line as `town_a,town_b,km`.

Vertices and adjacency list

ID	Town	Region	Degree	Neighbours (distance in km)
0	Chitipa	North	2	Karonga (100), Rumphi (230)
1	Karonga	North	2	Chitipa (100), Rumphi (165)
2	Rumphi	North	3	Chitipa (230), Karonga (165), Mzuzu (67)
3	Mzuzu	North	3	Rumphi (67), Nkhata Bay (47), Mzimba (110)
4	Nkhata Bay	North	2	Mzuzu (47), Nkhotakota (230)
5	Mzimba	North	2	Mzuzu (110), Kasungu (140)
6	Kasungu	Central	5	Mzimba (140), Nkhotakota (120), Ntchisi (90), Lilongwe (127), Mchinji (140)
7	Nkhotakota	Central	4	Nkhata Bay (230), Kasungu (120), Ntchisi (70), Salima (110)
8	Ntchisi	Central	3	Kasungu (90), Nkhotakota (70), Dowa (50)
9	Lilongwe	Central	5	Kasungu (127), Dowa (55), Salima (103), Mchinji (110), Dedza (85)
10	Mchinji	Central	2	Kasungu (140), Lilongwe (110)
11	Salima	Central	5	Nkhotakota (110), Dowa (85), Lilongwe (103), Dedza (125), Mangochi (190)
12	Dowa	Central	3	Ntchisi (50), Lilongwe (55), Salima (85)
13	Dedza	Central	3	Lilongwe (85), Salima (125), Ntcheu (75)
14	Ntcheu	Central	3	Dedza (75), Balaka (60), Blantyre (160)
15	Mangochi	South	2	Salima (190), Liwonde (70)
16	Balaka	South	3	Ntcheu (60), Liwonde (30), Blantyre (110)
17	Blantyre	South	7	Ntcheu (160), Balaka (110), Zomba (65), Mwanza (95), Thyolo (45), Mulanje (70), Chikwawa (50)

ID	Town	Region	Degree	Neighbours (distance in km)
18	Liwonde	South	3	Balaka (30), Mangochi (70), Zomba (55)
19	Zomba	South	3	Liwonde (55), Blantyre (65), Mulanje (100)
20	Mulanje	South	3	Zomba (100), Blantyre (70), Thyolo (45)
21	Mwanza	South	1	Blantyre (95)
22	Thyolo	South	3	Blantyre (45), Mulanje (45), Nsanje (150)
23	Chikwawa	South	2	Blantyre (50), Nsanje (125)
24	Nsanje	South	2	Chikwawa (125), Thyolo (150)

Edges (weights in km)

Edge	Town A	Town B	km
e1	Chitipa	Karonga	100
e2	Chitipa	Rumphi	230
e3	Karonga	Rumphi	165
e4	Rumphi	Mzuzu	67
e5	Mzuzu	Nkhata Bay	47
e6	Mzuzu	Mzimba	110
e7	Mzimba	Kasungu	140
e8	Nkhata Bay	Nkhotakota	230
e9	Kasungu	Nkhotakota	120
e10	Kasungu	Ntchisi	90
e11	Kasungu	Lilongwe	127
e12	Kasungu	Mchinji	140
e13	Nkhotakota	Ntchisi	70
e14	Nkhotakota	Salima	110
e15	Ntchisi	Dowa	50
e16	Dowa	Lilongwe	55
e17	Dowa	Salima	85
e18	Lilongwe	Salima	103
e19	Lilongwe	Mchinji	110

Edge	Town A	Town B	km
e20	Lilongwe	Dedza	85
e21	Dedza	Salima	125
e22	Dedza	Ntcheu	75
e23	Salima	Mangochi	190
e24	Ntcheu	Balaka	60
e25	Ntcheu	Blantyre	160
e26	Balaka	Liwonde	30
e27	Balaka	Blantyre	110
e28	Liwonde	Mangochi	70
e29	Liwonde	Zomba	55
e30	Zomba	Blantyre	65
e31	Zomba	Mulanje	100
e32	Blantyre	Mwanza	95
e33	Blantyre	Thyolo	45
e34	Blantyre	Mulanje	70
e35	Thyolo	Mulanje	45
e36	Blantyre	Chikwawa	50
e37	Chikwawa	Nsanje	125
e38	Thyolo	Nsanje	150

2. Drawing and description of the graph

Q2 · 10 marks

Conventions used. Each town is drawn as a circle (vertex) labelled with its name. Each road is a straight line (edge) between two circles. Because the graph is undirected, the lines have no arrowheads. The number in the box on each line is the weight, the distance in km. Towns are placed roughly where they are on the map of Malawi (north at the top), with Lake Malawi shown for orientation only; it is not part of the graph.

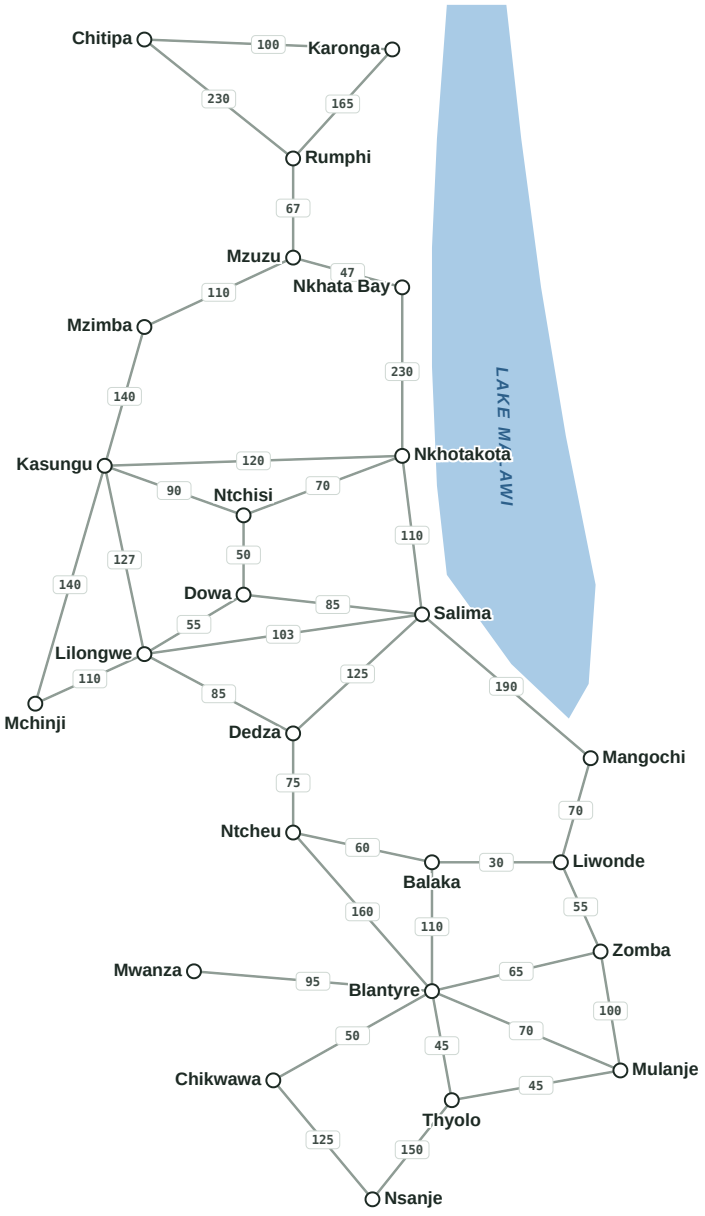


Figure 1. The Malawi road network graph, G = (V, E), |V| = 25, |E| = 38.

Description

- **Connected:** there is a path between every pair of towns, so G has a single component. Test T10 confirms this: all 625 source–target pairs have a finite distance.
- **Sparse:** a simple graph on 25 vertices can have at most 25 × 24 / 2 = 300 edges. G has 38, a density of 38 / 300 ≈ 0.13. This is typical of road networks and is why an adjacency list is the right representation.

- **Degrees:** the sum of degrees is $2|E| = 76$ (handshake lemma), so the average degree is 3.04. Blantyre has the highest degree (7), as the commercial hub of the south. Kasungu, Lilongwe and Salima have degree 5. Mwanza has degree 1 (a pendant vertex on the Mozambique border road), so every route to Mwanza must pass through Blantyre.
- **Cycles:** G contains many cycles, for example Blantyre–Thyolo–Mulanje–Blantyre (45 + 45 + 70 = 160 km) and Lilongwe–Dowa–Salima–Lilongwe. The cyclomatic number $|E| - |V| + 1 = 14$ is the number of independent cycles, and also the number of roads that can be removed while keeping all towns connected (see the MST in section 6).
- **Bridges:** Blantyre–Mwanza is a bridge (removing it disconnects Mwanza). The northern towns join the rest of the country through two corridors: the M1 through Mzimba and Kasungu, and the M5 lakeshore road through Nkhata Bay and Nkhotakota.
- **Longest and shortest roads:** Chitipa–Rumphi and Nkhata Bay–Nkhotakota (230 km) are the longest. Balaka–Liwonde (30 km) is the shortest.

> The assignment asks for the graph to be drawn on paper. Figure 1 is the typed version; the hand-drawn version follows the same layout and conventions.

3. How Dijkstra's algorithm works on the graph

Q3 · 15 marks

Dijkstra's algorithm finds the shortest distance from one **source** vertex to every other vertex in a graph with non-negative weights. It keeps a tentative distance *d[v]* for every town, starting at 0 for the source and ∞ for the rest. At each step it **settles** the unsettled town with the smallest tentative distance; that distance is now final, because every other route would have to leave through a town that is already at least as far away and the weights are never negative. It then **relaxes** each road (u, v) out of the settled town: if d[u] + w(u, v) < d[v], it lowers d[v] and records u as the previous town on v's route.

```
DIJKSTRA(G, s)
  for each vertex v:  d[v] ← ∞,  prev[v] ← NIL
  d[s] ← 0;  Q ← min-priority queue containing (0, s)
  while Q is not empty:
      (du, u) ← EXTRACT-MIN(Q)
      if u is already settled: continue          // stale entry
      mark u settled
      for each edge (u, v, w):
          if v not settled and du + w < d[v]:
              d[v] ← du + w;  prev[v] ← u;  INSERT(Q, (d[v], v))
```

Worked example: all shortest routes from Lilongwe

We run the algorithm with Lilongwe as the source. The table shows, for each of the 25 steps, which town is settled, the roads that improved a tentative distance, and the queue of reached-but-unsettled towns afterwards (in order of distance). A town leaves the queue in exactly the order of its final distance.

Step	Settle	d	Relaxations (old → new)	Tentative distances after the step
1	**Lilongwe**	0	Kasungu: ∞ → **127** Dowa: ∞ → **55** Salima: ∞ → **103** Mchinji: ∞ → **110** Dedza: ∞ → **85**	Dowa 55, Dedza 85, Salima 103, Mchinji 110, Kasungu 127
2	**Dowa**	55	Ntchisi: ∞ → **105**	Dedza 85, Salima 103, Ntchisi 105, Mchinji 110, Kasungu 127
3	**Dedza**	85	Ntcheu: ∞ → **160**	Salima 103, Ntchisi 105, Mchinji 110, Kasungu 127, Ntcheu 160
4	**Salima**	103	Nkhotakota: ∞ → **213** Mangochi: ∞ → **293**	Ntchisi 105, Mchinji 110, Kasungu 127, Ntcheu 160, Nkhotakota 213, Mangochi 293
5	**Ntchisi**	105	Nkhotakota: 213 → **175**	Mchinji 110, Kasungu 127, Ntcheu 160, Nkhotakota 175, Mangochi 293
6	**Mchinji**	110	none (all neighbours settled or no shorter)	Kasungu 127, Ntcheu 160, Nkhotakota 175, Mangochi 293
7	**Kasungu**	127	Mzimba: ∞ → **267**	Ntcheu 160, Nkhotakota 175, Mzimba 267, Mangochi 293
8	**Ntcheu**	160	Balaka: ∞ → **220** Blantyre: ∞ → **320**	Nkhotakota 175, Balaka 220, Mzimba 267, Mangochi 293, Blantyre 320
9	**Nkhotakota**	175	Nkhata Bay: ∞ → **405**	Balaka 220, Mzimba 267, Mangochi 293, Blantyre 320, Nkhata Bay 405
10	**Balaka**	220	Liwonde: ∞ → **250**	Liwonde 250, Mzimba 267, Mangochi 293, Blantyre 320, Nkhata Bay 405

Step	Settle	d	Relaxations (old → new)	Tentative distances after the step
11	**Liwonde**	250	Zomba: ∞ → **305**	Mzimba 267, Mangochi 293, Zomba 305, Blantyre 320, Nkhata Bay 405
12	**Mzimba**	267	Mzuzu: ∞ → **377**	Mangochi 293, Zomba 305, Blantyre 320, Mzuzu 377, Nkhata Bay 405
13	**Mangochi**	293	none (all neighbours settled or no shorter)	Zomba 305, Blantyre 320, Mzuzu 377, Nkhata Bay 405
14	**Zomba**	305	Mulanje: ∞ → **405**	Blantyre 320, Mzuzu 377, Nkhata Bay 405, Mulanje 405
15	**Blantyre**	320	Mwanza: ∞ → **415** Thyolo: ∞ → **365** Mulanje: 405 → **390** Chikwawa: ∞ → **370**	Thyolo 365, Chikwawa 370, Mzuzu 377, Mulanje 390, Nkhata Bay 405, Mwanza 415
16	**Thyolo**	365	Nsanje: ∞ → **515**	Chikwawa 370, Mzuzu 377, Mulanje 390, Nkhata Bay 405, Mwanza 415, Nsanje 515
17	**Chikwawa**	370	Nsanje: 515 → **495**	Mzuzu 377, Mulanje 390, Nkhata Bay 405, Mwanza 415, Nsanje 495
18	**Mzuzu**	377	Rumphi: ∞ → **444**	Mulanje 390, Nkhata Bay 405, Mwanza 415, Rumphi 444, Nsanje 495
19	**Mulanje**	390	none (all neighbours settled or no shorter)	Nkhata Bay 405, Mwanza 415, Rumphi 444, Nsanje 495
20	**Nkhata Bay**	405	none (all neighbours settled or no shorter)	Mwanza 415, Rumphi 444, Nsanje 495
21	**Mwanza**	415	none (all neighbours settled or no shorter)	Rumphi 444, Nsanje 495
22	**Rumphi**	444	Chitipa: ∞ → **674** Karonga: ∞ → **609**	Nsanje 495, Karonga 609, Chitipa 674
23	**Nsanje**	495	none (all neighbours settled or no shorter)	Karonga 609, Chitipa 674
24	**Karonga**	609	none (all neighbours settled or no shorter)	Chitipa 674
25	**Chitipa**	674	none (all neighbours settled or no shorter)	(empty)

Some steps show the algorithm correcting itself. In step 4, Salima sets Nkhotakota to 103 + 110 = 213 km. In step 5, Ntchisi (105 km) finds a shorter road, 105 + 70 = 175 km, so Nkhotakota is updated to 175 and its previous town changes to Ntchisi. In step 14, Zomba sets Mulanje to 405 km, and in step 15 Blantyre improves it to 320 + 70 = 390 km.

Result: shortest distances and routes from Lilongwe

Town	Distance (km)	Previous town	Shortest route
Lilongwe	0	–	Lilongwe
Dowa	55	Lilongwe	Lilongwe → Dowa
Dedza	85	Lilongwe	Lilongwe → Dedza
Salima	103	Lilongwe	Lilongwe → Salima
Ntchisi	105	Dowa	Lilongwe → Dowa → Ntchisi
Mchinji	110	Lilongwe	Lilongwe → Mchinji

Town	Distance (km)	Previous town	Shortest route
Kasungu	127	Lilongwe	Lilongwe → Kasungu
Ntcheu	160	Dedza	Lilongwe → Dedza → Ntcheu
Nkhotakota	175	Ntchisi	Lilongwe → Dowa → Ntchisi → Nkhotakota
Balaka	220	Ntcheu	Lilongwe → Dedza → Ntcheu → Balaka
Liwonde	250	Balaka	Lilongwe → Dedza → Ntcheu → Balaka → Liwonde
Mzimba	267	Kasungu	Lilongwe → Kasungu → Mzimba
Mangochi	293	Salima	Lilongwe → Salima → Mangochi
Zomba	305	Liwonde	Lilongwe → Dedza → Ntcheu → Balaka → Liwonde → Zomba
Blantyre	320	Ntcheu	Lilongwe → Dedza → Ntcheu → Blantyre
Thyolo	365	Blantyre	Lilongwe → Dedza → Ntcheu → Blantyre → Thyolo
Chikwawa	370	Blantyre	Lilongwe → Dedza → Ntcheu → Blantyre → Chikwawa
Mzuzu	377	Mzimba	Lilongwe → Kasungu → Mzimba → Mzuzu
Mulanje	390	Blantyre	Lilongwe → Dedza → Ntcheu → Blantyre → Mulanje
Nkhata Bay	405	Nkhotakota	Lilongwe → Dowa → Ntchisi → Nkhotakota → Nkhata Bay
Mwanza	415	Blantyre	Lilongwe → Dedza → Ntcheu → Blantyre → Mwanza
Rumphi	444	Mzuzu	Lilongwe → Kasungu → Mzimba → Mzuzu → Rumphi
Nsanje	495	Chikwawa	Lilongwe → Dedza → Ntcheu → Blantyre → Chikwawa → Nsanje
Karonga	609	Rumphi	Lilongwe → Kasungu → Mzimba → Mzuzu → Rumphi → Karonga
Chitipa	674	Rumphi	Lilongwe → Kasungu → Mzimba → Mzuzu → Rumphi → Chitipa

The previous-town pointers form the **shortest-path tree** rooted at Lilongwe (Figure 2). Reading the route to any town means following the pointers back from that town to Lilongwe. For example, Blantyre ← Ntcheu ← Dedza ← Lilongwe gives Lilongwe → Dedza → Ntcheu → Blantyre, 85 + 75 + 160 = 320 km. Note that this is shorter than going via Balaka (85 + 75 + 60 + 110 = 330 km) or via Salima, Mangochi and Liwonde.

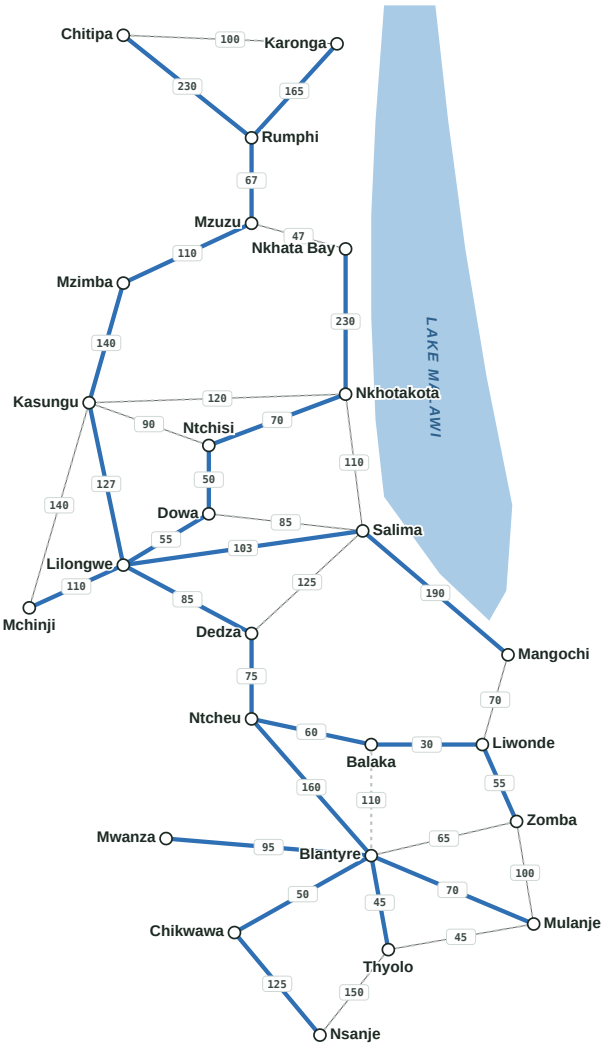


Figure 2. Shortest-path tree from Lilongwe (blue). Dashed roads are not used by any shortest route from Lilongwe.

4. Implementation

Q4 · 10 marks

The algorithm is implemented in **C++17** with only the standard library. The repository is organised as follows:

File	Purpose
`data/malawi_roads.csv`	The graph: one road per line, `town_a,town_b,km`.
`src/graph.hpp`	Graph class (adjacency list), Dijkstra, path reconstruction, Kruskal (with union–find) and Prim.
`src/main.cpp`	Command line program: shortest route, step-by-step trace, distances from a town, MST, and an interactive menu.
`tests/tests.cpp`	25 automated tests (section 5).
`docs/`	Web version of the same algorithms in JavaScript, published with GitHub Pages (section 8).
`Makefile`	`make` builds everything, `make test` runs the tests.

Design decisions

- **Adjacency list** (`vector<vector<pair<int, long long>>>`): uses O(V + E) memory and lets Dijkstra visit only the real neighbours of a town. An adjacency matrix would need 25 × 25 = 625 cells for 38 roads.
- **Town names are mapped to integer IDs** with a `std::map`, so the algorithms work on fast array indices while the user types names.
- **Binary heap priority queue** (`std::priority_queue` with `std::greater`) holding (distance, town) pairs. Instead of a decrease-key operation, a town is pushed again when its distance improves, and outdated entries are skipped when popped ("lazy deletion"). This keeps the code short and gives **O((V + E) log V)** time.
- **Path reconstruction** uses the `prev[]` array, following pointers back from the target and reversing.
- **Input validation:** negative weights are rejected (Dijkstra is incorrect with them), as are self loops, malformed lines, non-numeric distances, missing files and unknown town names. Unreachable towns are reported as "No route".

Core code: Dijkstra

```
struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest km from source (INF if unreachable)
    std::vector<int> prev;        // prev[v] = previous town on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                  // town removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (town, old dist, new dist)
};

// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
```

```
    using Item = std::pair<long long, int>;  // (distance, town)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

Building and running

```
$ make                       # builds ./roads and ./run_tests
$ ./roads path Lilongwe Blantyre
Shortest route Lilongwe -> Blantyre: 320 km
  Lilongwe -> Dedza -> Ntcheu -> Blantyre
$ ./roads trace Mzuzu Salima     # step-by-step dry run
$ ./roads all Zomba              # distances from Zomba to every town
$ ./roads mst kruskal            # minimum spanning tree
$ ./roads                        # interactive menu
$ make test                      # run the 25 tests
```

5. Testing and dry run

Q5 · 20 marks

Test strategy

The tests in `tests/tests.cpp` cover four groups: (a) loading the graph, (b) known shortest routes worked out by hand, (c) an independent cross-check of every pair of towns against a different algorithm (Floyd–Warshall), and (d) error handling and edge cases. Each test prints its input, the expected output and the actual output. The table below was produced by running `make test`.

Summary: 25 passed, 0 failed

ID	Test	Input	Expected	Actual	Result
T01	town and road count	data/malawi_roads.csv	`25 towns, 38 roads`	`25 towns, 38 roads`	PASS
T02	same start and end	Lilongwe → Lilongwe	`0 km: Lilongwe`	`0 km: Lilongwe`	PASS
T03	direct neighbours	Lilongwe → Dedza	`85 km: Lilongwe > Dedza`	`85 km: Lilongwe > Dedza`	PASS
T04	capital to commercial city	Lilongwe → Blantyre	`320 km: Lilongwe > Dedza > Ntcheu > Blantyre`	`320 km: Lilongwe > Dedza > Ntcheu > Blantyre`	PASS
T05	indirect route beats direct road	Ntcheu → Zomba (Ntcheu > Blantyre > Zomba would be 225)	`145 km: Ntcheu > Balaka > Liwonde > Zomba`	`145 km: Ntcheu > Balaka > Liwonde > Zomba`	PASS
T06	northern tip to southern tip	Chitipa → Nsanje	`1169 km: Chitipa > Rumphi > Mzuzu > Mzimba > Kasungu > Lilongwe > Dedza > Ntcheu > Blantyre > Chikwawa > Nsanje`	`1169 km: Chitipa > Rumphi > Mzuzu > Mzimba > Kasungu > Lilongwe > Dedza > Ntcheu > Blantyre > Chikwawa > Nsanje`	PASS
T07	symmetric (undirected)	Nsanje → Chitipa	`1169 km`	`1169 km`	PASS
T08	lakeshore route	Nkhata Bay → Salima	`340 km: Nkhata Bay > Nkhotakota > Salima`	`340 km: Nkhata Bay > Nkhotakota > Salima`	PASS
T09	north to lake south	Mzuzu → Mangochi	`577 km: Mzuzu > Nkhata Bay > Nkhotakota > Salima > Mangochi`	`577 km: Mzuzu > Nkhata Bay > Nkhotakota > Salima > Mangochi`	PASS
T10	all pairs agree, all paths add up	every source and target	`0 mismatches`	`0 mismatches`	PASS
T11	unknown town	Lilongwe → Kampala	`error: Unknown town: Kampala`	`error: Unknown town: Kampala`	PASS
T12	town names are case sensitive	lilongwe → Dedza	`error: Unknown town: lilongwe`	`error: Unknown town: lilongwe`	PASS
T13	negative distance rejected	addEdge(A, B, -5)	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	PASS

ID	Test	Input	Expected	Actual	Result
T14	non-numeric distance in file	Lilongwe,Dedza,eighty	`error: Line 1: bad distance 'eighty'`	`error: Line 1: bad distance 'eighty'`	PASS
T15	wrong separator in file	Lilongwe;Dedza;85	`error: Line 1: expected town_a,town_b,km`	`error: Line 1: expected town_a,town_b,km`	PASS
T16	missing data file	no_such_file.csv	`error: Cannot open no_such_file.csv`	`error: Cannot open no_such_file.csv`	PASS
T17	unreachable town (island)	Lilongwe → Likoma	`unreachable`	`unreachable`	PASS
T18	MST of disconnected graph is a forest	{Lilongwe-Dedza 85, Likoma-Chizumulu 10}	`2 edges, total 95, spanning=false`	`2 edges, total 95, spanning=false`	PASS
T19	zero-weight road allowed	A-B 0, B-C 4, A-C 9; A → C	`4 km: A > B > C`	`4 km: A > B > C`	PASS
T20	parallel roads: shorter one used	A-B 10 and A-B 3; A → B	`3 km: A > B`	`3 km: A > B`	PASS
T21	Kruskal MST size	Malawi graph	`24 edges, spanning=true`	`24 edges, spanning=true`	PASS
T22	Kruskal MST total	Malawi graph	`1889 km`	`1889 km`	PASS
T23	Prim gives the same total	Malawi graph, start at Chitipa	`1889 km`	`1889 km`	PASS
T24	Prim from every start town	25 different start towns	`0 mismatches`	`0 mismatches`	PASS
T25	MST connects all towns	Kruskal edges	`1 component`	`1 component`	PASS

Errors found during testing

When the tests were first run, **5 of 25 failed** (T04, T06, T07, T09, T22). In every case the program was right and our hand calculation was wrong:

- **T04 Lilongwe → Blantyre.** We expected the route through Balaka (330 km, which we had also mis-added as 305). The program found the direct Ntcheu–Blantyre road: 85 + 75 + 160 = 320 km.
- **T06/T07 Chitipa → Nsanje.** We expected the route to start Chitipa → Karonga → Rumphi (100 + 165 = 265 km) and missed the direct Chitipa–Rumphi road (230 km), which is 35 km shorter. The correct total is 1169 km.
- **T09 Mzuzu → Mangochi.** The route was right but we added the distances wrongly: 47 + 230 + 110 + 190 = 577 km, not 600.
- **T22 MST total.** We guessed the total before doing the Kruskal trace. The trace in section 7 gives 1889 km, and Prim gives the same.

We trusted the program over our arithmetic only after test T10 showed that Dijkstra agrees with Floyd–Warshall for all 625 pairs and that every reconstructed path adds up to its reported distance. We then

re-did the hand calculations and corrected the expected values. This shows why it is useful to test against an independent method rather than only against hand-worked examples.

Error cases (T11–T16) check that the program fails with a clear message instead of crashing: an unknown town (“Kampala”), wrong capitalisation, a negative distance, a non-numeric distance, the wrong separator and a missing data file. Edge cases (T17–T20) use small separate graphs: an island town with no road (Likoma) is reported as unreachable, a zero-length road is allowed, and when two roads join the same towns the shorter one is used.

Dry run 1: Dedza → Zomba, tracing the priority queue

This dry run follows the code line by line, showing the real contents of the priority queue, including the stale entries left behind by lazy deletion. Entries are written (town, distance) and the queue is shown sorted, smallest first.

Iter.	Action	Relaxing each road of the settled town	Priority queue after
init	–	dist[Dedza] = 0, all others ∞	(Dedza, 0)
1	pop (Dedza, 0) settle Dedza	Lilongwe: 0 + 85 = 85 < ∞ → update, prev = Dedza Salima: 0 + 125 = 125 < ∞ → update, prev = Dedza Ntcheu: 0 + 75 = 75 < ∞ → update, prev = Dedza	(Ntcheu, 75) (Lilongwe, 85) (Salima, 125)
2	pop (Ntcheu, 75) settle Ntcheu	Dedza: settled, skip Balaka: 75 + 60 = 135 < ∞ → update, prev = Ntcheu Blantyre: 75 + 160 = 235 < ∞ → update, prev = Ntcheu	(Lilongwe, 85) (Salima, 125) (Balaka, 135) (Blantyre, 235)
3	pop (Lilongwe, 85) settle Lilongwe	Kasungu: 85 + 127 = 212 < ∞ → update, prev = Lilongwe Dowa: 85 + 55 = 140 < ∞ → update, prev = Lilongwe Salima: 85 + 103 = 188 ≥ 125 → no change Mchinji: 85 + 110 = 195 < ∞ → update, prev = Lilongwe Dedza: settled, skip	(Salima, 125) (Balaka, 135) (Dowa, 140) (Mchinji, 195) (Kasungu, 212) (Blantyre, 235)
4	pop (Salima, 125) settle Salima	Nkhotakota: 125 + 110 = 235 < ∞ → update, prev = Salima Dowa: 125 + 85 = 210 ≥ 140 → no change Lilongwe: settled, skip Dedza: settled, skip Mangochi: 125 + 190 = 315 < ∞ → update, prev = Salima	(Balaka, 135) (Dowa, 140) (Mchinji, 195) (Kasungu, 212) (Nkhotakota, 235) (Blantyre, 235) (Mangochi, 315)
5	pop (Balaka, 135) settle Balaka	Ntcheu: settled, skip Liwonde: 135 + 30 = 165 < ∞ → update, prev = Balaka Blantyre: 135 + 110 = 245 ≥ 235 → no change	(Dowa, 140) (Liwonde, 165) (Mchinji, 195) (Kasungu, 212) (Nkhotakota, 235) (Blantyre, 235) (Mangochi, 315)

Iter.	Action	Relaxing each road of the settled town	Priority queue after
6	pop (Dowa, 140) settle Dowa	Ntchisi: 140 + 50 = 190 < ∞ → update, prev = Dowa Lilongwe: settled, skip Salima: settled, skip	(Liwonde, 165) (Ntchisi, 190) (Mchinji, 195) (Kasungu, 212) (Nkhotakota, 235) (Blantyre, 235) (Mangochi, 315)
7	pop (Liwonde, 165) settle Liwonde	Balaka: settled, skip Mangochi: 165 + 70 = 235 < 315 → update, prev = Liwonde Zomba: 165 + 55 = 220 < ∞ → update, prev = Liwonde	(Ntchisi, 190) (Mchinji, 195) (Kasungu, 212) (Zomba, 220) (Nkhotakota, 235) (Mangochi, 235) (Blantyre, 235) (Mangochi, 315)
8	pop (Ntchisi, 190) settle Ntchisi	Kasungu: 190 + 90 = 280 ≥ 212 → no change Nkhotakota: 190 + 70 = 260 ≥ 235 → no change Dowa: settled, skip	(Mchinji, 195) (Kasungu, 212) (Zomba, 220) (Nkhotakota, 235) (Mangochi, 235) (Blantyre, 235) (Mangochi, 315)
9	pop (Mchinji, 195) settle Mchinji	Kasungu: 195 + 140 = 335 ≥ 212 → no change Lilongwe: settled, skip	(Kasungu, 212) (Zomba, 220) (Nkhotakota, 235) (Mangochi, 235) (Blantyre, 235) (Mangochi, 315)
10	pop (Kasungu, 212) settle Kasungu	Mzimba: 212 + 140 = 352 < ∞ → update, prev = Kasungu Nkhotakota: 212 + 120 = 332 ≥ 235 → no change Ntchisi: settled, skip Lilongwe: settled, skip Mchinji: settled, skip	(Zomba, 220) (Nkhotakota, 235) (Mangochi, 235) (Blantyre, 235) (Mangochi, 315) (Mzimba, 352)
11	pop (Zomba, 220) settle Zomba	Liwonde: settled, skip Blantyre: 220 + 65 = 285 ≥ 235 → no change Mulanje: 220 + 100 = 320 < ∞ → update, prev = Zomba **Target settled: stop.**	(Nkhotakota, 235) (Mangochi, 235) (Blantyre, 235) (Mangochi, 315) (Mulanje, 320) (Mzimba, 352)

Result: following prev[] back from Zomba gives Dedza → Ntcheu → Balaka → Liwonde → Zomba, a distance of **220 km**. The program prints the same:

```
$ ./roads path Dedza Zomba
Shortest route Dedza -> Zomba: 220 km
  Dedza -> Ntcheu -> Balaka -> Liwonde -> Zomba
```

Dry run 2: program trace for Lilongwe → Blantyre

The `trace` command prints the same steps the algorithm takes. It matches steps 1–15 of the hand run in section 3.

```
$ ./roads trace Lilongwe Blantyre
Dry run of Dijkstra from Lilongwe
Step  1: settle Lilongwe (0 km)
          update Kasungu: inf -> 127
          update Dowa: inf -> 55
          update Salima: inf -> 103
          update Mchinji: inf -> 110
          update Dedza: inf -> 85
Step  2: settle Dowa (55 km)
```

```
          update Ntchisi: inf -> 105
Step  3: settle Dedza (85 km)
          update Ntcheu: inf -> 160
Step  4: settle Salima (103 km)
          update Nkhotakota: inf -> 213
          update Mangochi: inf -> 293
Step  5: settle Ntchisi (105 km)
          update Nkhotakota: 213 -> 175
Step  6: settle Mchinji (110 km), no updates
Step  7: settle Kasungu (127 km)
          update Mzimba: inf -> 267
Step  8: settle Ntcheu (160 km)
          update Balaka: inf -> 220
          update Blantyre: inf -> 320
Step  9: settle Nkhotakota (175 km)
          update Nkhata Bay: inf -> 405
Step 10: settle Balaka (220 km)
          update Liwonde: inf -> 250
Step 11: settle Liwonde (250 km)
          update Zomba: inf -> 305
Step 12: settle Mzimba (267 km)
          update Mzuzu: inf -> 377
Step 13: settle Mangochi (293 km), no updates
Step 14: settle Zomba (305 km)
          update Mulanje: inf -> 405
Step 15: settle Blantyre (320 km)
          update Mwanza: inf -> 415
          update Thyolo: inf -> 365
          update Mulanje: 405 -> 390
          update Chikwawa: inf -> 370
Target Blantyre settled, so its distance is final.

Shortest route Lilongwe -> Blantyre: 320 km
  Lilongwe -> Dedza -> Ntcheu -> Blantyre
```

6. Does the graph have a minimum spanning tree?

Q6 · 10 marks

Yes. A spanning tree is a subset of the edges that connects all the vertices with no cycles; it has exactly |V| − 1 edges. A minimum spanning tree (MST) is a spanning tree with the smallest possible total weight. An undirected graph has a spanning tree if and only if it is **connected**, and then it also has a minimum one, because there are finitely many spanning trees and one of them must have the smallest total. Our graph is undirected, weighted and connected (section 2), so it has an MST.

The MST of our graph has **24 edges** and a total length of **1889 km**, compared with 3,854 km for all 38 roads. The other 14 roads (the cyclomatic number from section 2) each close a cycle and are not needed for connectivity.

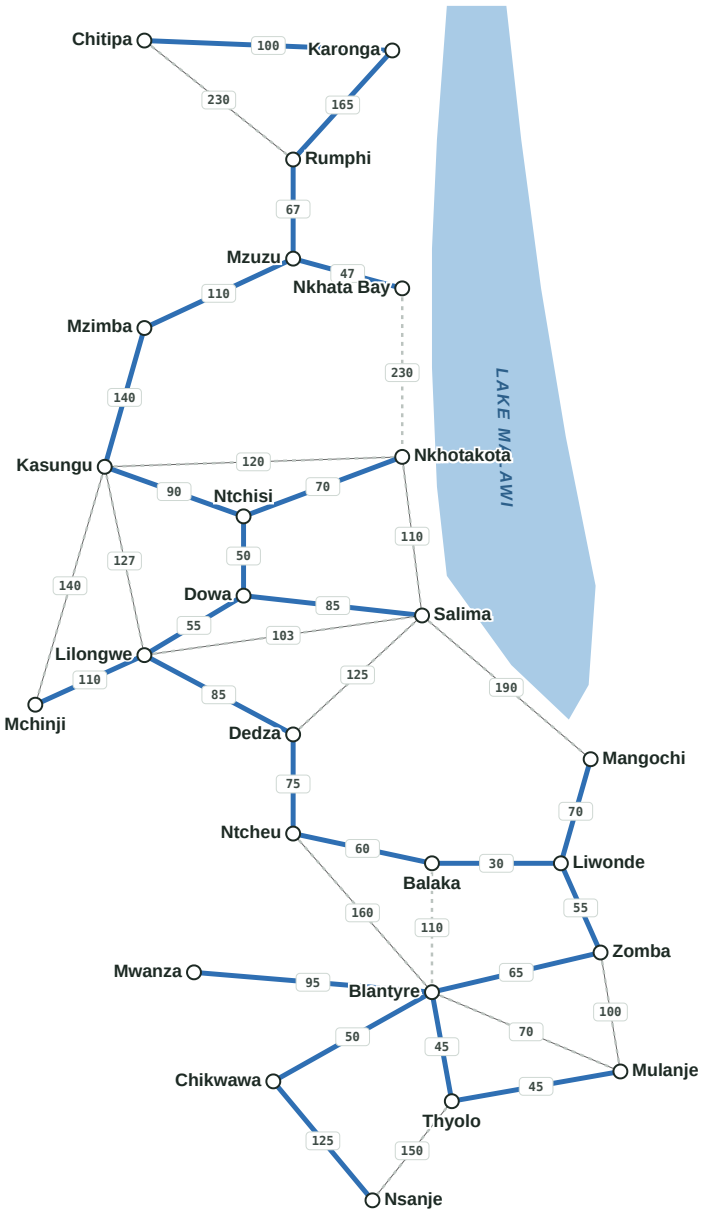


Figure 3. Minimum spanning tree (blue, 1889 km). Dashed roads are not in the MST.

The MST is unique. Some roads have equal lengths (for example two roads of 45 km and two of 85 km), which can sometimes allow more than one MST. We checked every road that is not in the tree: in each case it is strictly longer than every road on the tree path between its two towns, so no swap gives an equal total. The MST is therefore the only one.

What the MST represents

The MST is the **cheapest set of roads that still connects every town** to every other town. In the Malawi context it answers questions such as:

- **Road maintenance on a limited budget.** If the Roads Authority could only keep 24 roads in good condition, the MST keeps all 25 district centres reachable for the least total kilometres of road to maintain.
- **Laying infrastructure along roads.** ESCOM power lines, fibre-optic cable or water pipelines are usually laid along road reserves. Connecting all 25 towns along the MST needs 1889 km of cable instead of 3,854 km.
- **Emergency and relief logistics.** During floods in the Lower Shire (Chikwawa, Nsanje), the MST shows a minimal backbone that must stay open to keep every town connected.

What the MST does not represent: it is not a set of shortest routes. In the MST the only route from Lilongwe to Blantyre is Lilongwe → Dedza → Ntcheu → Balaka → Liwonde → Zomba → Blantyre, which is **370 km**, compared with the 320 km found by Dijkstra. The MST minimises the total length of the whole network; Dijkstra minimises the length of one journey.

7. Which MST algorithm we would use, and why

Q7 · 10 marks

	Kruskal	Prim
Idea	Sort all roads by length. Add each road unless it joins two towns that are already connected (it would make a cycle).	Start from one town. Repeatedly add the shortest road from the tree to a town not yet in the tree.
Data structure	Sorted edge list + union–find (disjoint sets)	Priority queue + adjacency list
Time	O(E log E)	O(E log V) with a binary heap; O(V^2) with an array
Best for	Sparse graphs, data given as an edge list	Dense graphs, data given as an adjacency matrix
On our graph	38 × $\log_2 38$ ≈ 200 basic steps	38 × $\log_2 25$ ≈ 176 basic steps

We would use Kruskal's algorithm, for these reasons:

1. **The graph is sparse** (38 edges, density 0.13). Kruskal's cost depends on sorting the edges, which is cheap when there are few of them. The O(V^2) array version of Prim, which suits dense graphs, would waste work here. For a graph this small the binary-heap versions of both algorithms are practically equal in speed, so the decision comes down to the next points.
2. **Our data is already an edge list.** The file `malawi_roads.csv` is a list of roads with lengths, which is exactly Kruskal's input. No adjacency structure is needed.
3. **It is easy to explain and check by hand.** Kruskal processes roads from shortest to longest and makes one yes/no decision for each, as in the trace below. This makes it easy to verify, and it matches how a planner would think: “build the cheapest links first, unless the towns are already connected”.
4. **It handles a disconnected network gracefully.** If floods cut the network into pieces (test T18), Kruskal returns a minimum spanning forest, one tree per piece, without any change. Prim only builds the tree of the piece containing its start town.

We implemented **both** algorithms to confirm the answer. Prim gives the same 1889 km total from every one of the 25 possible start towns (test T24).

Kruskal trace on our graph

#	Road (sorted by length)	km	Decision
1	Balaka – Liwonde	30	add
2	Blantyre – Thyolo	45	add
3	Thyolo – Mulanje	45	add
4	Mzuzu – Nkhata Bay	47	add
5	Ntchisi – Dowa	50	add
6	Blantyre – Chikwawa	50	add
7	Dowa – Lilongwe	55	add
8	Liwonde – Zomba	55	add
9	Ntcheu – Balaka	60	add

#	Road (sorted by length)	km	Decision
10	Zomba – Blantyre	65	add
11	Rumphi – Mzuzu	67	add
12	Nkhotakota – Ntchisi	70	add
13	Liwonde – Mangochi	70	add
14	Blantyre – Mulanje	70	skip: Blantyre and Mulanje already connected (cycle)
15	Dedza – Ntcheu	75	add
16	Dowa – Salima	85	add
17	Lilongwe – Dedza	85	add
18	Kasungu – Ntchisi	90	add
19	Blantyre – Mwanza	95	add
20	Chitipa – Karonga	100	add
21	Zomba – Mulanje	100	skip: Zomba and Mulanje already connected (cycle)
22	Lilongwe – Salima	103	skip: Lilongwe and Salima already connected (cycle)
23	Mzuzu – Mzimba	110	add
24	Nkhotakota – Salima	110	skip: Nkhotakota and Salima already connected (cycle)
25	Lilongwe – Mchinji	110	add
26	Balaka – Blantyre	110	skip: Balaka and Blantyre already connected (cycle)
27	Kasungu – Nkhotakota	120	skip: Kasungu and Nkhotakota already connected (cycle)
28	Dedza – Salima	125	skip: Dedza and Salima already connected (cycle)
29	Chikwawa – Nsanje	125	add
30	Kasungu – Lilongwe	127	skip: Kasungu and Lilongwe already connected (cycle)
31	Mzimba – Kasungu	140	add
32	Kasungu – Mchinji	140	skip: Kasungu and Mchinji already connected (cycle)
33	Thyolo – Nsanje	150	skip: Thyolo and Nsanje already connected (cycle)
34	Ntcheu – Blantyre	160	skip: Ntcheu and Blantyre already connected (cycle)
35	Karonga – Rumphi	165	add
36	Salima – Mangochi	190	skip: Salima and Mangochi already connected (cycle)
37	Chitipa – Rumphi	230	skip: Chitipa and Rumphi already connected (cycle)
38	Nkhata Bay – Nkhotakota	230	skip: Nkhata Bay and Nkhotakota already connected (cycle)

The tree is complete after 24 additions (the last is Karonga–Rumphi, 165 km). The algorithm still scans the remaining longer roads but none can be added. Total: **1889 km**.

Prim's trace from Lilongwe (for comparison)

#	Cheapest road leaving the tree	km
1	Lilongwe → Dowa	55
2	Dowa → Ntchisi	50
3	Ntchisi → Nkhotakota	70
4	Lilongwe → Dedza	85
5	Dedza → Ntcheu	75
6	Ntcheu → Balaka	60

#	Cheapest road leaving the tree	km
7	Balaka → Liwonde	30
8	Liwonde → Zomba	55
9	Zomba → Blantyre	65
10	Blantyre → Thyolo	45
11	Thyolo → Mulanje	45
12	Blantyre → Chikwawa	50
13	Liwonde → Mangochi	70
14	Dowa → Salima	85
15	Ntchisi → Kasungu	90
16	Blantyre → Mwanza	95
17	Lilongwe → Mchinji	110
18	Chikwawa → Nsanje	125
19	Kasungu → Mzimba	140
20	Mzimba → Mzuzu	110
21	Mzuzu → Nkhata Bay	47
22	Mzuzu → Rumphi	67
23	Rumphi → Karonga	165
24	Karonga → Chitipa	100

Total: **1889 km**, the same tree as Kruskal, built in a different order.

8. Deployment

Q8 · 10 marks

The work is deployed in two ways, so it can be tested with nothing to install.

Web application (no installation)

The same Dijkstra, Kruskal and Prim algorithms are written in JavaScript in `docs/graph.js` and run in the browser. The page is hosted free on GitHub Pages:

https://mse23-mkapolo-cpu.github.io/DSA/

- **Shortest route:** choose any two towns. The page shows the distance, the route highlighted on the map, and the full step-by-step table of the algorithm with the queue after each step.
- **Spanning tree:** shows the MST on the map with the Kruskal decision table, or Prim's order.
- **Tests:** runs the same test cases as the C++ suite in the browser and shows pass or fail for each.

C++ program

The C++ source is on GitHub at https://github.com/mse23-mkapolo-cpu/DSA. It can be run without installing anything by opening the repository in GitHub Codespaces (Code → Codespaces → Create), then running `make && ./roads` in the terminal. Alternatively, `src/graph.hpp` and `src/main.cpp` can be pasted into an online C++ compiler such as OnlineGDB. On a machine with g++ the steps are:

```
git clone https://github.com/mse23-mkapolo-cpu/DSA.git
cd DSA
make && make test
./roads path Lilongwe Blantyre
```

Appendix A: src/graph.hpp

```
// graph.hpp - Weighted undirected graph of Malawi towns, with
// Dijkstra's shortest path and two Minimum Spanning Tree algorithms
// (Prim and Kruskal).
#pragma once

#include <algorithm>
#include <fstream>
#include <functional>
#include <limits>
#include <map>
#include <queue>
#include <sstream>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>

namespace mw {

const long long INF = std::numeric_limits<long long>::max() / 4;

struct Edge {
    int u, v;
    long long w;
};

class Graph {
public:
    // Add a town if it is new; return its index either way.
    int addNode(const std::string& name) {
        auto it = index_.find(name);
        if (it != index_.end()) return it->second;
        int id = static_cast<int>(names_.size());
        names_.push_back(name);
        index_[name] = id;
        adj_.emplace_back();
        return id;
    }

    void addEdge(const std::string& a, const std::string& b, long long w) {
        if (w < 0) throw std::invalid_argument("Negative distance on road " + a + " - " + b +
                                               ": Dijkstra needs non-negative weights");
        if (a == b) throw std::invalid_argument("Self loop on " + a + " is not allowed");
        int u = addNode(a), v = addNode(b);
        adj_[u].push_back({v, w});
        adj_[v].push_back({u, w});
        edges_.push_back({u, v, w});
    }

    // Load "town_a,town_b,km" lines. Blank lines and lines starting with # are skipped.
    static Graph fromCsv(std::istream& in) {
        Graph g;
        std::string line;
        int lineNo = 0;
        while (std::getline(in, line)) {
            ++lineNo;
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (line.empty() || line[0] == '#') continue;
            std::stringstream ss(line);
            std::string a, b, w;
            if (!std::getline(ss, a, ',') || !std::getline(ss, b, ',') || !std::getline(ss, w)) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": expected town_a,town_b,km");
            }
            long long km;
            try {
                size_t used = 0;
                km = std::stoll(w, &used);
                if (used != w.size()) throw std::invalid_argument("trailing");
            } catch (const std::exception&) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": bad distance '" + w + "'");
            }
            g.addEdge(a, b, km);
        }
        return g;
```

```
    }

    static Graph fromFile(const std::string& path) {
        std::ifstream f(path);
        if (!f) throw std::runtime_error("Cannot open " + path);
        return fromCsv(f);
    }

    int size() const { return static_cast<int>(names_.size()); }
    const std::vector<Edge>& edges() const { return edges_; }
    const std::string& name(int id) const { return names_.at(id); }
    const std::vector<std::pair<int, long long>>& neighbours(int u) const { return adj_.at(u); }

    int id(const std::string& name) const {
        auto it = index_.find(name);
        if (it == index_.end()) throw std::out_of_range("Unknown town: " + name);
        return it->second;
    }
    bool has(const std::string& name) const { return index_.count(name) > 0; }

private:
    std::vector<std::string> names_;
    std::map<std::string, int> index_;
    std::vector<std::vector<std::pair<int, long long>>> adj_;
    std::vector<Edge> edges_;
};

// ------------------------------------------------------------------ Dijkstra

struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest km from source (INF if unreachable)
    std::vector<int> prev;        // prev[v] = previous town on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                  // town removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (town, old dist, new dist)
};

// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
    using Item = std::pair<long long, int>;  // (distance, town)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
```

```
    return path;
}

// ------------------------------------------------------------- MST

struct MstResult {
    std::vector<Edge> edges;
    long long total = 0;
    bool spanning = false;  // false if the graph is disconnected (then it is a forest)
};

// Disjoint Set Union with path compression and union by rank.
class DSU {
public:
    explicit DSU(int n) : parent_(n), rank_(n, 0) {
        for (int i = 0; i < n; ++i) parent_[i] = i;
    }
    int find(int x) { return parent_[x] == x ? x : parent_[x] = find(parent_[x]); }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
        parent_[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }

private:
    std::vector<int> parent_, rank_;
};

// Kruskal: sort edges by weight, add each edge that joins two different components. O(E log E).
inline MstResult kruskal(const Graph& g) {
    MstResult r;
    std::vector<Edge> es = g.edges();
    std::stable_sort(es.begin(), es.end(), [](const Edge& a, const Edge& b) { return a.w < b.w; });
    DSU dsu(g.size());
    for (const Edge& e : es) {
        if (dsu.unite(e.u, e.v)) {
            r.edges.push_back(e);
            r.total += e.w;
        }
    }
    r.spanning = g.size() > 0 && static_cast<int>(r.edges.size()) == g.size() - 1;
    return r;
}

// Prim: grow one tree from `start`, always taking the cheapest edge leaving it. O(E log V).
inline MstResult prim(const Graph& g, int start = 0) {
    MstResult r;
    int n = g.size();
    if (n == 0) return r;
    std::vector<bool> inTree(n, false);
    using Item = std::tuple<long long, int, int>;  // (weight, from, to)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;
    pq.push({0, -1, start});
    while (!pq.empty()) {
        auto [w, from, to] = pq.top();
        pq.pop();
        if (inTree[to]) continue;
        inTree[to] = true;
        if (from != -1) {
            r.edges.push_back({from, to, w});
            r.total += w;
        }
        for (auto [v, wv] : g.neighbours(to))
            if (!inTree[v]) pq.push({wv, to, v});
    }
    r.spanning = static_cast<int>(r.edges.size()) == n - 1;
    return r;
}

}  // namespace mw
```

Appendix B: src/main.cpp

```
// main.cpp - Command line program for the Malawi road network.
//
// Usage:
//   roads [--data FILE] path   <From> <To>    shortest route between two towns
//   roads [--data FILE] trace  <From> <To>    same, plus a step-by-step dry run of Dijkstra
//   roads [--data FILE] all    <From>         shortest distance to every town
//   roads [--data FILE] mst    [prim|kruskal] minimum spanning tree
//   roads [--data FILE] towns                 list the towns
//   roads [--data FILE]                       interactive menu
#include <iomanip>
#include <iostream>

#include "graph.hpp"

using namespace mw;

static std::string distStr(long long d) { return d >= INF ? "inf" : std::to_string(d); }

static void printPath(const Graph& g, const std::string& from, const std::string& to, bool withTrace) {
    int s = g.id(from), t = g.id(to);
    std::vector<TraceStep> trace;
    DijkstraResult r = dijkstra(g, s, withTrace ? &trace : nullptr);

    if (withTrace) {
        std::cout << "Dry run of Dijkstra from " << from << "\n";
        int step = 1;
        for (const TraceStep& st : trace) {
            std::cout << "Step " << std::setw(2) << step++ << ": settle " << g.name(st.settled)
                      << " (" << st.settledDist << " km)";
            if (st.relaxed.empty()) std::cout << ", no updates";
            for (auto [v, oldD, newD] : st.relaxed)
                std::cout << "\n          update " << g.name(v) << ": " << distStr(oldD) << " -> " << newD;
            std::cout << "\n";
            if (st.settled == t) {
                std::cout << "Target " << to << " settled, so its distance is final.\n";
                break;
            }
        }
        std::cout << "\n";
    }

    std::vector<int> path = buildPath(r, t);
    if (path.empty()) {
        std::cout << "No route from " << from << " to " << to << "\n";
        return;
    }
    std::cout << "Shortest route " << from << " -> " << to << ": " << r.dist[t] << " km\n";
    for (size_t i = 0; i < path.size(); ++i) {
        std::cout << (i ? " -> " : "  ") << g.name(path[i]);
    }
    std::cout << "\n";
}

static void printAll(const Graph& g, const std::string& from) {
    DijkstraResult r = dijkstra(g, g.id(from));
    std::cout << "Shortest distances from " << from << ":\n";
    for (int v = 0; v < g.size(); ++v) {
        std::cout << "  " << std::left << std::setw(12) << g.name(v) << std::right << std::setw(6)
                  << distStr(r.dist[v]) << " km   via ";
        std::vector<int> p = buildPath(r, v);
        for (size_t i = 0; i < p.size(); ++i) std::cout << (i ? " > " : "") << g.name(p[i]);
        std::cout << "\n";
    }
}

static void printMst(const Graph& g, const std::string& algo) {
    MstResult m;
    if (algo == "prim") m = prim(g);
    else if (algo == "kruskal") m = kruskal(g);
    else throw std::invalid_argument("MST algorithm must be prim or kruskal");
    std::cout << "Minimum spanning tree (" << algo << "), in the order edges were added:\n";
    for (const Edge& e : m.edges)
        std::cout << "  " << std::left << std::setw(12) << g.name(e.u) << " - " << std::setw(12) << g.name(e.v)
                  << std::right << std::setw(5) << e.w << " km\n";
    std::cout << "Edges: " << m.edges.size() << ", total length: " << m.total << " km"
              << (m.spanning ? "" : "  (graph is disconnected: this is a spanning forest)") << "\n";
}
```

```
static void printTowns(const Graph& g) {
    std::cout << g.size() << " towns, " << g.edges().size() << " roads\n";
    for (int v = 0; v < g.size(); ++v) std::cout << "  " << g.name(v) << "\n";
}

static void interactive(const Graph& g) {
    while (true) {
        std::cout << "\n1) Shortest route  2) Route with dry run  3) Distances from a town\n"
                     "4) MST (Prim)  5) MST (Kruskal)  6) List towns  0) Quit\nChoice: ";
        std::string c;
        if (!std::getline(std::cin, c) || c == "0") return;
        try {
            std::string a, b;
            if (c == "1" || c == "2" || c == "3") {
                std::cout << "From town: ";
                std::getline(std::cin, a);
            }
            if (c == "1" || c == "2") {
                std::cout << "To town: ";
                std::getline(std::cin, b);
            }
            if (c == "1") printPath(g, a, b, false);
            else if (c == "2") printPath(g, a, b, true);
            else if (c == "3") printAll(g, a);
            else if (c == "4") printMst(g, "prim");
            else if (c == "5") printMst(g, "kruskal");
            else if (c == "6") printTowns(g);
            else std::cout << "Unknown choice\n";
        } catch (const std::exception& e) {
            std::cout << "Error: " << e.what() << "\n";
        }
    }
}

int main(int argc, char** argv) {
    std::vector<std::string> args(argv + 1, argv + argc);
    std::string data = "data/malawi_roads.csv";
    if (args.size() >= 2 && args[0] == "--data") {
        data = args[1];
        args.erase(args.begin(), args.begin() + 2);
    }
    try {
        Graph g = Graph::fromFile(data);
        if (args.empty()) interactive(g);
        else if (args[0] == "path" && args.size() == 3) printPath(g, args[1], args[2], false);
        else if (args[0] == "trace" && args.size() == 3) printPath(g, args[1], args[2], true);
        else if (args[0] == "all" && args.size() == 2) printAll(g, args[1]);
        else if (args[0] == "mst") printMst(g, args.size() > 1 ? args[1] : "kruskal");
        else if (args[0] == "towns") printTowns(g);
        else {
            std::cerr << "Usage: roads [--data FILE] path|trace <From> <To> | all <From> | mst [prim|kruskal] |
towns\n";
            return 2;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 1;
    }
    return 0;
}
```

Appendix C: MST program output

```
$ ./roads mst kruskal
Minimum spanning tree (kruskal), in the order edges were added:
  Balaka       - Liwonde        30 km
  Blantyre     - Thyolo         45 km
  Thyolo       - Mulanje        45 km
  Mzuzu        - Nkhata Bay     47 km
  Ntchisi      - Dowa           50 km
  Blantyre     - Chikwawa       50 km
  Dowa         - Lilongwe       55 km
  Liwonde      - Zomba          55 km
  Ntcheu       - Balaka         60 km
```

```
  Zomba        - Blantyre       65 km
  Rumphi       - Mzuzu          67 km
  Nkhotakota   - Ntchisi        70 km
  Liwonde      - Mangochi       70 km
  Dedza        - Ntcheu         75 km
  Dowa         - Salima         85 km
  Lilongwe     - Dedza          85 km
  Kasungu      - Ntchisi        90 km
  Blantyre     - Mwanza         95 km
  Chitipa      - Karonga       100 km
  Mzuzu        - Mzimba        110 km
  Lilongwe     - Mchinji       110 km
  Chikwawa     - Nsanje        125 km
  Mzimba       - Kasungu       140 km
  Karonga      - Rumphi        165 km
Edges: 24, total length: 1889 km
```